SLOT 11

FORMS & CONTROLLERS – VALIDATION & ERRORS – SUBMIT FLOW & DATA

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Forms & Controllers
- Validation & Errors
- Submit Flow & Data

1 NHÓM 1: FORMS & CONTROLLERS (NỀN TẢNG NHẬP LIỆU)

1. Form trong Flutter là gì?

Khái niệm quan trọng

`Form` là widget quản lý tập hợp nhiều field nhập liệu
Kết hợp với `GlobalKey<FormState>`
Dùng để validate & submit tập trung

Mục tiêu bài học

Hiểu vì sao không nên validate từng TextField riêng lẻ
Biết cách tổ chức form có cấu trúc
Làm chủ `FormState`

Ý nghĩa thực tế:
Mọi app thật đều có login, register, search, filter, profile edit…

2. TextField vs TextFormField

Khái niệm quan trọng

`TextField`: chỉ nhập liệu
`TextFormField`: tích hợp sẵn validation cho Form

Mục tiêu bài học

Biết chọn widget đúng ngữ cảnh
Không viết validation thủ công rời rạc

3. TextEditingController

Khái niệm quan trọng

Điều khiển & đọc giá trị text
Có thể setText, clear, listen thay đổi

Mục tiêu bài học

Hiểu vòng đời controller
Biết dispose đúng cách
Không lạm dụng controller khi không cần

Sai lầm phổ biến:
Dùng controller cho mọi field dù chỉ cần validator

4. Focus & FocusNode (cơ bản)

Khái niệm quan trọng

Điều khiển focus giữa các input
Di chuyển sang field tiếp theo

Mục tiêu bài học

Tạo UX nhập liệu mượt
Hiểu submit từ bàn phím

PhucPT10@fpt.edu.vn

NHÓM 2: VALIDATION & ERROR HANDLING

5. Validation trong Form

Khái niệm quan trọng

- `validator` function
- Trả về `String?` (null = hợp lệ)

Mục tiêu bài học

- Viết validation rõ ràng
- Không validate trong onPressed
- Hiểu validate theo rule (email, password…)

6. FormState & validate()

Khái niệm quan trọng

- `_formKey.currentState!.validate()`
- Chỉ submit khi hợp lệ

Mục tiêu bài học

- Kiểm soát submit flow
- Không gửi dữ liệu khi form sai

7. Error Message & UX

Khái niệm quan trọng

- errorText
- hiển thị lỗi thân thiện
- không gây rối layout

Mục tiêu bài học

- Thiết kế UX hợp lý khi báo lỗi
- Tránh layout nhảy khi hiển thị error

8. Real-time validation vs Submit-time validation

Khái niệm quan trọng

- Validate khi gõ
- Validate khi bấm submit

Mục tiêu bài học

- Biết chọn chiến lược phù hợp
- Không gây khó chịu cho user

NHÓM 3: SUBMIT FLOW & DATA HANDLING

9. Submit Flow chuẩn trong Flutter

Khái niệm quan trọng

- Validate → Save → Process → Feedback
- Disable button khi đang submit

Mục tiêu bài học

- Tạo flow chuyên nghiệp
- Không cho submit nhiều lần

10. Lấy & xử lý dữ liệu Form

Khái niệm quan trọng

PhucPT10@fpt.edu.vn

Đọc từ controller
Hoặc dùng onSaved()

Mục tiêu bài học

Hiểu hai cách lấy dữ liệu
Biết tổ chức dữ liệu trước khi gửi API

◈ 11. Loading & Feedback sau submit

Khái niệm quan trọng

CircularProgressIndicator
SnackBar / Dialog
Thành công / thất bại

Mục tiêu bài học

UX đầy đủ
Không để user "bấm mà không biết gì"

◈ 12. Reset & Clear Form

Khái niệm quan trọng

`_formKey.currentState!.reset()`
controller.clear()

Mục tiêu bài học

Làm sạch form sau submit
Tránh dữ liệu cũ còn sót

IV NHÓM 4: TƯ DUY THỰC TẾ KHI LÀM FORM

◈ 13. Form Structure & Maintainability

Khái niệm quan trọng

Tách widget nhỏ
Không viết form 400 dòng

Mục tiêu bài học

Dễ mở rộng
Dễ refactor

◈ 14. Common Mistakes

Quên dispose controller
Validate nhưng vẫn submit
Không xử lý keyboard che input
Layout vỡ khi có error text

Mục tiêu bài học

Tránh bug phổ biến
Viết form "giống app thật"

🎯 KẾT QUẢ SAU SLOT 11

Sau bài này, sinh viên sẽ:

Hiểu cấu trúc form chuẩn trong Flutter
Biết viết validation đúng cách
Thiết kế submit flow chuyên nghiệp
Sẵn sàng làm:
    Login screen

PhucPT10@fpt.edu.vn

Register screen
Edit profile
Search / Filter form

👉 Đây là bước nền cho:
API Integration
State Management
Authentication

📚 TÀI LIỆU THAM KHẢO UY TÍN

1 Flutter Official Documentation (Forms)
https://docs.flutter.dev/cookbook/forms/validation
Tài liệu chính thức, chuẩn và cập nhật

2 Flutter Complete Reference – Alberto Miola (Sách)
Giải thích rõ cấu trúc Form & lifecycle
Phù hợp sinh viên muốn hiểu sâu

3 Flutter Official YouTube – Form Validation Video
Kênh chính thức của Flutter
Demo thực tế, dễ theo dõi

4 Reso Coder – Flutter Form Validation Tutorial
Phân tích chi tiết validation logic
Phù hợp sinh viên trung cấp

5 FilledStacks (YouTube & Blog)
Hướng dẫn cách tổ chức form trong app thật
Liên hệ với state management

PhucPT10@fpt.edu.vn

FORM STRUCTURE & ARCHITECTURE (NỀN TẢNG BẮT BUỘC)

Nếu không hiểu phần này, mọi validation phía sau sẽ rời rạc và khó kiểm soát.

1. Hiểu đúng vai trò của `Form` & `FormState`

Khái niệm quan trọng

`Form` là container quản lý nhiều field
`GlobalKey<FormState>` dùng để điều khiển form

Kỹ thuật cần thành thạo

Tạo và dùng `_formKey`
Gọi `.validate()` đúng thời điểm
Biết khi nào dùng `.save()` và `.reset()`

Lỗi thường gặp

Không dùng Form, validate thủ công từng field
Không dùng GlobalKey → không kiểm soát được form

Best Practice

Mỗi màn hình nhập liệu → luôn có `Form`
Không submit nếu `validate()` trả về false

2. Phân biệt `TextField` và `TextFormField`

Khái niệm quan trọng

`TextField`: nhập liệu cơ bản
`TextFormField`: hỗ trợ validation

Kỹ thuật cần thành thạo

Dùng `validator` đúng cách
Kết hợp với `Form`

Lỗi thường gặp

Dùng TextField rồi tự viết errorText thủ công
Không tận dụng validator

Best Practice

Trong form → mặc định dùng `TextFormField`

CONTROLLERS & DATA HANDLING

Đây là phần giúp sinh viên hiểu dữ liệu đi từ đâu đến đâu.

3. Làm chủ `TextEditingController`

Khái niệm quan trọng

Controller đọc & thay đổi giá trị input
Cần `dispose()` trong StatefulWidget

Kỹ thuật cần thành thạo

Lấy giá trị: `controller.text`
Clear field
Lắng nghe thay đổi

PhucPT10@fpt.edu.vn

Dispose đúng lifecycle

⚠ Lỗi thường gặp

Quên dispose → memory leak
Dùng controller khi chỉ cần validator

✅ Best Practice

Chỉ dùng controller khi cần đọc hoặc thao tác text
Form lớn → quản lý controller rõ ràng

4. Focus & Keyboard Flow

Khái niệm quan trọng

FocusNode
textInputAction

Kỹ thuật cần thành thạo

Chuyển focus sang field tiếp theo
Submit bằng bàn phím

⚠ Lỗi thường gặp

Không xử lý keyboard che input
UX nhập liệu kém

✅ Best Practice

Tối ưu flow nhập liệu giống app thật (login, register)

I VALIDATION – LOGIC BẮT BUỘC PHẢI LÀM ĐÚNG

Validation không chỉ là kiểm tra rỗng. Đây là kiểm soát dữ liệu đầu vào.

5. Viết `validator()` chuẩn

Khái niệm quan trọng

Trả về `String?`
null = hợp lệ

Kỹ thuật cần thành thạo

Check rỗng
Check độ dài
Check định dạng (email, password)
Tách logic validation rõ ràng

⚠ Lỗi thường gặp

Validate trong onPressed
Không xử lý trim()
Thông báo lỗi không rõ ràng

✅ Best Practice

Validator gọn, rõ ràng
Message dễ hiểu, thân thiện

6. Validation Flow chuyên nghiệp

Khái niệm quan trọng

validate() trước khi submit
Không cho submit khi form sai

PhucPT10@fpt.edu.vn

🛠 Kỹ thuật cần thành thạo

```
if (_formKey.currentState!.validate()) {
  // submit
}
```

⚠ Lỗi thường gặp

Gọi API dù form sai
Không disable nút submit khi loading

✅ Best Practice

Validate → Submit → Loading → Feedback

IV ERROR HANDLING & UX

Form không chỉ đúng logic, mà phải có UX tốt.

7. Hiển thị lỗi hợp lý

📌 Khái niệm quan trọng

errorText
Layout không bị nhảy

🛠 Kỹ thuật cần thành thạo

maxLines cho Text
Thiết kế spacing hợp lý

⚠ Lỗi thường gặp

Layout vỡ khi xuất hiện error
Text tràn

✅ Best Practice

Dự phòng không gian cho error
Không làm UI thay đổi mạnh khi có lỗi

8. Real-time vs Submit-time validation

📌 Khái niệm quan trọng

Validate khi gõ
Validate khi bấm submit

🛠 Kỹ thuật cần thành thạo

AutovalidateMode

⚠ Lỗi thường gặp

Validate quá sớm → gây khó chịu
Không validate gì cả

✅ Best Practice

Login đơn giản → validate khi submit
Form dài → có thể validate realtime hợp lý

V SUBMIT FLOW & DATA PROCESSING

Đây là phần mô phỏng hành vi app thật.

9. Submit Flow chuẩn production

📌 Khái niệm quan trọng

Không submit nhiều lần

PhucPT10@fpt.edu.vn

Có trạng thái loading

🛠 Kỹ thuật cần thành thạo

Disable button khi loading
Hiển thị CircularProgressIndicator
SnackBar / Dialog phản hồi

⚠ Lỗi thường gặp

User bấm 3–4 lần
Không có phản hồi khi submit

✅ Best Practice

Submit = 1 flow rõ ràng, không mơ hồ

◈ 10. Reset & Clear Form đúng cách

📌 Khái niệm quan trọng

reset()
clear()

🛠 Kỹ thuật cần thành thạo

Reset form sau submit thành công
Không xóa khi submit thất bại

⚠ Lỗi thường gặp

Xóa dữ liệu dù API lỗi
Giữ dữ liệu cũ sau thành công

✅ Best Practice

Reset có điều kiện

VI. TƯ DUY CHUYÊN NGHIỆP PHẢI CÓ SAU SLOT 11

◈ 11. Không viết Form 400 dòng trong 1 widget

🛠 Kỹ thuật

Tách widget nhỏ
Tách validation logic

✅ Best Practice

Form rõ cấu trúc
Dễ maintain

◈ 12. Hiểu dữ liệu đi từ đâu đến đâu

Sinh viên phải hiểu:
Input → Controller → Validate → Submit → API → Response → Feedback
Nếu không hiểu flow này → chưa đủ nền tảng cho:

Authentication
State management
Clean architecture

🎓 CHECKLIST – SAU SLOT 11 SINH VIÊN PHẢI LÀM ĐƯỢC

✓ Tạo Login Form hoàn chỉnh
✓ Viết validator chuẩn
✓ Quản lý controller đúng lifecycle

PhucPT10@fpt.edu.vn

✓ Xử lý submit flow chuyên nghiệp
✓ Không còn validate thủ công rời rạc
✓ Không còn quên dispose controller
✓ Không còn submit khi form sai

🎯 TÓM LẠI – 3 ĐIỀU QUAN TRỌNG NHẤT

Form phải có cấu trúc & kiểm soát tập trung
Validation là logic, không phải trang trí
Submit Flow phải giống app thật, không chỉ in ra console

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

I. TÌNH HUỐNG 1

🛒 Dự án E-Commerce – Form Đăng Ký / Đăng Nhập

🎬 Bối cảnh thực tế

Một ứng dụng bán hàng cần:

- Đăng ký tài khoản
- Đăng nhập
- Reset mật khẩu
- Cập nhật thông tin cá nhân

❗ Vấn đề gặp phải

- Người dùng nhập sai email
- Mật khẩu không đủ mạnh
- Submit nhiều lần gây duplicate request
- Không hiển thị lỗi rõ ràng → user bỏ app

🛠 Kỹ thuật được áp dụng

1 Forms & Controllers

- Mỗi field có `TextEditingController`
- Sử dụng `Form` + `GlobalKey<FormState>`
- Tách UI và logic xử lý submit

2 Validation

- Kiểm tra:
  - Email regex
  - Password length
  - Confirm password match
- Dùng:
  - `validator()` cho validation đồng bộ
  - Validation nâng cao khi gọi API (async validation)

3 Submit Flow

- Disable button khi đang loading
- Hiển thị CircularProgressIndicator
- Xử lý:
  - success → navigate
  - error → show message
- Tránh double submit

🎯 Mục tiêu khi áp dụng

- Tăng conversion rate
- Giảm lỗi backend
- Trải nghiệm mượt mà

📌 Bài học rút ra

- Validation phải thực hiện ở cả client & server
- Submit flow phải kiểm soát trạng thái loading
- Không bao giờ cho phép spam button

PhucPT10@fpt.edu.vn

II. TÌNH HUỐNG 2

🏢 Ứng dụng Ngân hàng – Form Chuyển Tiền

🎬 Bối cảnh thực tế

Form gồm:

Số tài khoản người nhận
Số tiền
Nội dung chuyển khoản
OTP xác thực

❗ Vấn đề thực tế

Nhập sai định dạng số tiền
Nhập số âm
Gửi request nhiều lần gây lỗi giao dịch
Xử lý lỗi không rõ ràng → user hoang mang

🛠 Cách áp dụng kỹ thuật

1 Validation nâng cao

Regex kiểm tra số tài khoản
Parse & kiểm tra số tiền > 0
Format tiền realtime
Hiển thị lỗi dưới field

2 Submit Flow nhiều bước

Flow chuyên nghiệp:

```
Validate local
→ Call API check account
→ Show confirm dialog
→ Submit transaction
→ OTP verify
→ Success screen
```

3 Error Handling

Phân loại:

- Network error
- Business error (không đủ tiền)
- OTP sai

Mapping error → hiển thị đúng message

🎯 Mục tiêu

Đảm bảo an toàn
Không xảy ra giao dịch lỗi
UX chuyên nghiệp

📌 Bài học rút ra

Submit Flow có thể là multi-step state machine
Không bao giờ xử lý transaction chỉ bằng 1 nút submit đơn giản

III. TÌNH HUỐNG 3

🏫 Hệ thống Quản lý Sinh viên – Form Thêm/Sửa Thông Tin

🎬 Bối cảnh

Admin có thể:

PhucPT10@fpt.edu.vn

Thêm sinh viên
Cập nhật thông tin
Import dữ liệu

❗ Vấn đề

Dữ liệu không đồng bộ
Trùng email
Không reset controller khi edit

🛠 Áp dụng kỹ thuật

1 Controller Lifecycle

Khởi tạo controller trong `initState`
Dispose trong `dispose`
Khi edit → setText từ model

2 Form Reuse

Cùng 1 form cho Create & Edit
Logic:
Nếu id == null → create
else → update

3 Clean Architecture

UI chỉ validate
Submit → gọi repository
Repository → call API

🎯 Mục tiêu

Code dễ maintain
Không duplicate form
Dễ mở rộng

📌 Bài học

Không trộn business logic vào widget
Form phải reusable

IV. TÌNH HUỐNG 4

🚚 Ứng dụng Giao Hàng – Form Nhiều Bước (Multi-Step Form)

🎬 Bối cảnh

Người dùng tạo đơn:
Chọn địa chỉ
Chọn phương thức giao
Thanh toán

❗ Vấn đề

Dữ liệu giữa các bước bị mất
Validate từng bước hay toàn bộ?
Quay lại bước trước mất dữ liệu

PhucPT10@fpt.edu.vn

🛠 Cách áp dụng

1 State Management

Lưu form data vào model chung
Không chỉ phụ thuộc controller

2 Partial Validation

Validate theo từng step
Hoặc validate cuối cùng

3 UX Optimization

Lưu tạm state
Confirm trước submit cuối

🎯 Mục tiêu

Tránh user mất dữ liệu
Trải nghiệm mượt

📌 Bài học

Form lớn → không xử lý như form nhỏ
Phải thiết kế data flow rõ ràng

V. DỰ ÁN THỰC TẾ – Startup EdTech

🎬 Tình huống

Startup xây dựng app thi thử online.
Form gồm:

Đăng ký
Nộp bài
Cập nhật profile

❗ Vấn đề họ gặp

Học sinh submit bài nhiều lần
Mất bài do reload
Không hiển thị lỗi rõ

🛠 Cách họ giải quyết

1 Disable submit khi loading

2 Lưu bài tạm thời (draft)

3 Async validation trước submit

4 Retry khi network fail

📊 Kết quả

Giảm 40% lỗi submit
Tăng 25% completion rate
Giảm support ticket

PhucPT10@fpt.edu.vn

TỔNG KẾT KẾT NỐI KIẾN THỨC – THỰC TIỄN

Kiến thức	Ứng dụng thực tế
TextEditingController	Quản lý input realtime
Form & GlobalKey	Validate toàn bộ form
validator()	Kiểm tra dữ liệu trước submit
Submit Flow	Điều khiển trạng thái chuyên nghiệp
Loading state	Ngăn double submit
Error mapping	UX rõ ràng
Multi-step form	Dự án quy mô lớn

Điều quan trọng sinh viên phải hiểu

Form không chỉ là UI — đó là data pipeline
Validation không chỉ để "đẹp" — mà để bảo vệ hệ thống
Submit flow là một state machine
Error handling quyết định UX
Dự án lớn luôn có:
- loading state
- retry
- error mapping
- separation of concerns

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

PHẦN 1 — FORMS & CONTROLLERS (CẤP ĐỘ CHUYÊN SÂU)

I. NỀN TẢNG: Form trong Flutter thực chất là gì?

Trong Flutter, `Form` không phải chỉ là container UI.
Nó là một **cơ chế quản lý trạng thái tập trung cho các FormField con**.
Bản chất:

- `Form` lưu trữ danh sách các `FormFieldState`
- `FormState.validate()` duyệt qua từng field và gọi validator
- `FormState.save()` gọi onSaved của từng field

👉 Nguyên lý cốt lõi:
Form = Composite Pattern (quản lý nhiều field như một đơn vị)

II. TextEditingController – Hiểu sâu

1 Bản chất

`TextEditingController` là một:

- ChangeNotifier
- giữ giá trị TextEditingValue

Nó lưu:

- text
- selection
- composing region (IME)

2 Vì sao phải dispose?

Controller giữ listener trong memory.
Nếu không dispose:

- Memory leak
- Lắng nghe text change không cần thiết

Production rule:

- Tạo trong initState
- Dispose trong dispose()

3 Khi nào KHÔNG nên dùng controller?

Trường hợp:

- Form nhỏ
- Không cần truy cập text realtime

Có thể dùng `onSaved()` thay vì controller.

👉 Trong dự án lớn:

- Chỉ dùng controller khi cần real-time processing
- Không dùng controller tràn lan

III. Các vấn đề thực tiễn với Controllers

1 Controller không sync với state

Sai lầm:

- Dùng controller + setState lung tung

Giải pháp:

- Hoặc controller
- Hoặc state management
- Không trộn lẫn không kiểm soát

PhucPT10@fpt.edu.vn

2 Edit Form Problem

Khi edit:

```
controller.text = user.name;
```

Nếu không set đúng thời điểm:

- Có thể override user input
- Có thể rebuild gây reset

Giải pháp production:

- Set value trong initState
- Không set trong build()

3 Rebuild Loop

Nếu addListener rồi setState bên trong:

- Có thể gây infinite rebuild

Phải kiểm soát:

- Debounce
- Throttling

PHẦN 2 — VALIDATION & ERROR (CẤP ĐỘ CHUYÊN SÂU)

I. Phân loại Validation

1 Client-side validation

- Required
- Regex
- Length
- Format

2 Server-side validation

- Email đã tồn tại?
- Username đã đăng ký?
- Không đủ tiền?

👉 Production rule:

KHÔNG BAO GIỜ tin hoàn toàn vào client validation.

II. Sync vs Async Validation

Sync Validation

```
validator: (value) {
  if (value!.isEmpty) return 'Required';
}
```

Chạy ngay lập tức.

Async Validation

Ví dụ:

- Check email tồn tại

Cách làm chuyên nghiệp:

- Không gọi API trong validator()
- Dùng debounce
- Lưu trạng thái async validation riêng

👉 Lý do:

validator() phải chạy nhanh

Không được block UI

PhucPT10@fpt.edu.vn

III. UX Pattern trong Validation

1 Validate on submit

Chỉ show lỗi khi bấm submit.

2 Validate realtime

Show lỗi ngay khi user gõ.

3 Hybrid

Sau lần submit đầu tiên → validate realtime

Production UX thường dùng hybrid.

IV. Error Mapping (Cực kỳ quan trọng)

Backend trả:

```
{
  "code": "EMAIL_EXISTS",
  "message": "Email already exists"
}
```

Không nên show raw message.

Phải:

Map code → message thân thiện

Có localization

V. Các vấn đề thực tế

❗ Spam API validation

Gõ mỗi ký tự → gọi API

Giải pháp:

Debounce 300–500ms

❗ Validation chậm gây lag

Giải pháp:

Không xử lý nặng trong validator

Tách logic

PHẦN 3 — SUBMIT FLOW & DATA (PRODUCTION LEVEL)

I. Submit Flow = State Machine

Submit thực tế không phải:

```
Click → API → Done
```

Mà là:

```
Idle
→ Validating
→ Loading
→ Success
→ Error
→ Retry
```

Đây là một finite state machine.

II. Kiến trúc chuẩn Production

UI Layer:

Form

Controller

Loading

PhucPT10@fpt.edu.vn

Business Layer:
- Validate logic nâng cao
- Submit use case

Data Layer:
- Repository
- API call

III. Double Submit Problem

User bấm 2 lần.
Hậu quả:
- Duplicate transaction
- Duplicate order

Giải pháp:
- Disable button khi loading
- Backend idempotency key

IV. Retry Strategy

Nếu network fail:
Có nên retry tự động?
Production thường:
- Retry 1–2 lần
- Hoặc cho user retry manual

V. Multi-step Submit

Trong dự án lớn:
Ví dụ:
- Validate local
- Upload file
- Submit data
- Confirm email

Submit không còn là 1 API.

PHẦN 4 — MULTI-STEP FORM (KIẾN TRÚC LỚN)

Vấn đề

- Giữ state giữa các step
- Validate từng phần
- Cho phép back

Cách triển khai chuyên nghiệp

1. Tạo FormData model chung
2. Mỗi step cập nhật model
3. Không phụ thuộc controller để lưu data
4. Chỉ controller để hiển thị UI

PHẦN 5 — CẬP NHẬT XU HƯỚNG HIỆN ĐẠI

PhucPT10@fpt.edu.vn

1 Reactive Form

Thay vì:
Form + setState
Dùng:
- State management (Bloc / Riverpod)
- Reactive streams

2 Form Architecture trong dự án lớn

Tách:
- Form Widget
- Form Validation Logic
- Submit UseCase
- Error Mapper

3 Accessibility

Production yêu cầu:
- Screen reader support
- Focus order
- Error message accessible

4 Internationalization (i18n)

Error message phải:
- Không hardcode
- Dùng localization

PHẦN 6 — NHỮNG SAI LẦM LỚN SINH VIÊN HAY MẮC

- Đặt toàn bộ logic trong widget
- Không dispose controller
- Gọi API trong validator
- Không disable submit
- Không xử lý loading state
- Không phân loại error

PHẦN 7 — TƯ DUY CỐT LÕI PHẢI ĐẠT

Sinh viên khá giỏi phải hiểu:
- Form là data pipeline
- Validation là defense layer
- Submit flow là state machine
- Controller là UI tool, không phải data storage chính
- Error handling là UX strategy

🎯 KẾT LUẬN

Sau khi hiểu sâu Slot 11 ở mức này, sinh viên có thể:
- ✅ Xây dựng form production-ready
- ✅ Thiết kế submit flow an toàn
- ✅ Xử lý validation chuyên nghiệp
- ✅ Tối ưu UX
- ✅ Áp dụng vào dự án ngân hàng, e-commerce, SaaS

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

PHẦN 1 — FORMS & CONTROLLERS

I. KINH NGHIỆM THỰC TẾ

1 Đừng dùng controller làm nơi lưu dữ liệu chính

❌ Sai tư duy:
Lấy dữ liệu từ controller khi cần
Xem controller như source of truth

✅ Production mindset:
Controller chỉ phục vụ UI
Data phải nằm trong model/state

👉 Lý do:
Controller bị dispose khi widget bị destroy
Không phù hợp cho multi-step form
Không testable

2 Form lớn phải tách module

Trong dự án lớn:
Không đặt 20 TextFormField trong 1 file
Không đặt toàn bộ validation trong widget

Best practice:
Tách:
FormSectionWidget
ValidationHelper
SubmitUseCase

3 Đừng set controller.text trong build()

Lỗi phổ biến:

```
build() {
  controller.text = user.name;
}
```

Hậu quả:
Reset input user
Gây loop rebuild

✔ Chỉ set trong initState hoặc didUpdateWidget.

II. LỖI PHỔ BIẾN

❗ Không dispose controller

Hậu quả:
Memory leak
Listener không bị remove
App chậm dần

❗ Sử dụng quá nhiều controller không cần thiết

Nếu form chỉ submit 1 lần:
→ Dùng onSaved()
Không cần controller cho mọi field.

PhucPT10@fpt.edu.vn

❗ Dùng setState trong addListener mà không kiểm soát

Dễ gây:
- Rebuild liên tục
- Giật lag

Giải pháp:
- Debounce
- Hoặc dùng state management

III. BEST PRACTICES

✅ Controller chỉ khi cần realtime
✅ Dispose đầy đủ
✅ Không business logic trong widget
✅ Không phụ thuộc controller làm source of truth
✅ Tách UI và Validation

PHẦN 2 — VALIDATION

I. KINH NGHIỆM THỰC TẾ

1 Validation là UX chiến lược, không chỉ kỹ thuật

Sai lầm của junior:
- Càng nhiều validation càng tốt

Thực tế:
- Validation quá sớm → user khó chịu
- Validation quá trễ → user bực bội

Production thường dùng:
Hybrid validation pattern:
- Chỉ validate sau lần submit đầu
- Sau đó validate realtime

2 Regex phức tạp dễ gây bug

Ví dụ:
- Email regex quá nghiêm
- Phone regex sai quốc gia

Best practice:
- Không tự viết regex quá phức tạp
- Dùng thư viện hoặc rule backend

3 Validation phải đồng bộ với backend

Nếu backend yêu cầu:
- Password có ký tự đặc biệt

Client cũng phải enforce.
Nếu không:
- Client pass
- Server reject

UX rất tệ.

II. LỖI PHỔ BIẾN

❗ Gọi API trong validator()

Không nên.

PhucPT10@fpt.edu.vn

Vì:
- validator phải sync
- Không được async block

❗ Hardcode error message

```
return "Email already exists";
```

Sai.
Nên:
- Map error code
- Support localization

❗ Không xử lý lỗi network

Validation không chỉ là format.
Phải xử lý:
- Timeout
- Server down
- No internet

III. BEST PRACTICES

✅ Validation nhẹ ở client
✅ Validation nghiêm ở server
✅ Map error code → user-friendly message
✅ Không xử lý async trong validator
✅ Debounce khi gọi API

PHẦN 3 — SUBMIT FLOW

I. KINH NGHIỆM THỰC TẾ

1 Submit Flow luôn có Loading State

Sai lầm phổ biến:
- Bấm submit → không hiển thị loading

User sẽ:
- Bấm lại
- Thoát app

2 Phải disable button khi loading

Không chỉ UI loading.
Phải:
- Disable button
- Hoặc ignore tap

3 Xử lý double submit ở backend

Frontend disable chưa đủ.
Backend nên:
- Có idempotency key
- Hoặc detect duplicate request

4 Không navigate ngay khi gọi API

Sai lầm:

PhucPT10@fpt.edu.vn

```
onPressed() {
  submit();
  Navigator.push(...);
}
```

Phải:

- Await API
- Check success
- Sau đó navigate

II. LỖI PHỔ BIẾN

❗ Không bắt exception

```
await repository.submit();
```

Nếu throw → crash.

Phải:

- Try-catch
- Emit error state

❗ Không phân loại lỗi

Hiển thị chung:

"Something went wrong"

Production nên phân loại:

- Network error
- Validation error
- Business error

❗ Không có retry strategy

Nếu network fail:

- App nên cho retry
- Không bắt user nhập lại từ đầu

III. BEST PRACTICES

✅ Submit = State machine
✅ Luôn có loading state
✅ Disable button khi loading
✅ Phân loại error rõ ràng
✅ Retry có kiểm soát
✅ Không navigate trước khi thành công

PHẦN 4 — MULTI-STEP FORM

I. KINH NGHIỆM THỰC TẾ

1 Không lưu state trong controller

Controller chết → mất dữ liệu.

Phải:

- Lưu vào model chung

2 Cho phép quay lại không mất dữ liệu

Sai lầm:

- Mỗi step rebuild → mất state

PhucPT10@fpt.edu.vn

Giải pháp:
- Shared ViewModel
- Hoặc State Management

II. LỖI PHỔ BIẾN

❗ Validate toàn bộ form khi user mới ở step 1
→ UX tệ
Phải:
- Validate từng step

III. BEST PRACTICES

✅ Centralized FormData
✅ Validate từng phần
✅ Confirm trước submit cuối
✅ Lưu draft khi form dài

PHẦN 5 — CÁC MẸO CHUYÊN NGHIỆP

💡 Mẹo 1: Luôn log submit

Trong production:
- Log thời gian submit
- Log failure reason

Giúp debug sau này.

💡 Mẹo 2: Không tin tưởng UI state

Backend phải validate lại toàn bộ.

💡 Mẹo 3: Tách error model riêng

Thay vì dùng String:
Tạo class:
- NetworkError
- ValidationError
- ServerError

💡 Mẹo 4: Form dài → auto-save draft

App thương mại điện tử thường:
- Lưu local draft
- Phục hồi khi mở lại

PHẦN 6 — CHECKLIST PRODUCTION READY

Sinh viên nên tự hỏi:
- Controller có dispose chưa?
- Có disable submit khi loading không?
- Có xử lý exception không?
- Có phân loại error không?
- Có double submit protection không?
- Có tách business logic khỏi widget không?
- Validation có đồng bộ backend không?
- Multi-step có lưu state tập trung không?

PhucPT10@fpt.edu.vn

Nếu tất cả là YES → gần production level.

TƯ DUY CUỐI CÙNG

Junior nghĩ:
Form là input + button.
Developer nghĩ:
Form là hệ thống kiểm soát dữ liệu, trạng thái, và rủi ro.
Senior nghĩ:
Form là một state machine có chiến lược UX và phòng thủ nhiều lớp.

PhucPT10@fpt.edu.vn

Smart Registration App

I. TÓM TẮT DỰ ÁN

Mô tả

Xây dựng một ứng dụng Flutter cho phép người dùng:

- Đăng ký tài khoản
- Validate dữ liệu
- Submit giả lập API call
- Xử lý loading
- Hiển thị lỗi
- Điều hướng sang màn hình thành công

Mục tiêu học tập

Sau dự án này, sinh viên có thể:

- Hiểu Form + GlobalKey hoạt động thế nào
- Sử dụng TextEditingController đúng cách
- Viết validator chuẩn
- Thiết kế submit flow có loading
- Xử lý lỗi và trạng thái
- Tránh double submit

II. CHỨC NĂNG CỦA ỨNG DỤNG

- Form gồm:
  - Họ tên
  - Email
  - Password
  - Confirm Password
- Validation:
  - Required
  - Email format
  - Password >= 6 ký tự
  - Confirm khớp password
- Submit Flow:
  - Validate
  - Loading 2 giây (giả lập API)
  - Nếu email = test@error.com → trả lỗi
  - Ngược lại → sang màn hình Success

III. CÔNG NGHỆ SỬ DỤNG

- Flutter SDK
- StatefulWidget
- TextEditingController
- Form + GlobalKey
- Future.delayed (giả API)
- Navigator

IV. CẤU TRÚC THƯ MỤC

```
lib/
 ├── main.dart
 ├── screens/
 │     ├── register_screen.dart
 │     └── success_screen.dart
 ├── models/
 │     └── user_model.dart
 ├── services/
 │     └── fake_auth_service.dart
 └── utils/
       └── validators.dart
```

V. TÓM TẮT CÁC BƯỚC THỰC HIỆN

Tạo project
Tạo cấu trúc thư mục
Tạo model User
Tạo validator helper
Tạo fake auth service
Xây dựng Register Screen
Tạo Submit Flow
Xây dựng Success Screen
Test các trường hợp lỗi
Kiểm tra double submit

VI. CHI TIẾT TỪNG BƯỚC

◈ BƯỚC 1 – TẠO PROJECT

```
flutter create smart_registration_app
cd smart_registration_app
```

Chạy thử:

```
flutter run
```

✅ Kiểm chứng: App chạy màn hình mặc định.

◈ BƯỚC 2 – TẠO CẤU TRÚC THƯ MỤC

Tạo các folder như trên.

✅ Kiểm chứng: Không lỗi import.

◈ BƯỚC 3 – TẠO USER MODEL

📁 models/user_model.dart

```
class UserModel {
  final String name;
  final String email;
  final String password;

  UserModel({
    required this.name,
    required this.email,
    required this.password,
  });
}
```

📌 Giải thích:

PhucPT10@fpt.edu.vn

Model dùng để tách dữ liệu khỏi UI
Không dùng controller làm data chính

BƯỚC 4 – TẠO VALIDATOR

utils/validators.dart

```
class Validators {
  static String? required(String? value) {
    if (value == null || value.trim().isEmpty) {
      return "Field is required";
    }
    return null;
  }

  static String? email(String? value) {
    if (value == null || value.isEmpty) return "Email required";
    final emailRegex =
        RegExp(r'^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$');
    if (!emailRegex.hasMatch(value)) {
      return "Invalid email format";
    }
    return null;
  }

  static String? password(String? value) {
    if (value == null || value.length < 6) {
      return "Password must be at least 6 characters";
    }
    return null;
  }
}
```

Kiểm chứng:
Gọi thử function trong debug

BƯỚC 5 – TẠO FAKE AUTH SERVICE

services/fake_auth_service.dart

```
import 'dart:async';
import '../models/user_model.dart';

class FakeAuthService {
  Future<void> register(UserModel user) async {
    await Future.delayed(const Duration(seconds: 2));

    if (user.email == "test@error.com") {
      throw Exception("Email already exists");
    }
  }
}
```

Giải thích:
Giả lập API
Dùng Future để mô phỏng async

BƯỚC 6 – XÂY DỰNG REGISTER SCREEN

screens/register_screen.dart

```
import 'package:flutter/material.dart';
import '../utils/validators.dart';
import '../models/user_model.dart';
import '../services/fake_auth_service.dart';
```


PhucPT10@fpt.edu.vn

```
import 'success_screen.dart';

class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _formKey = GlobalKey<FormState>();

  final _nameController = TextEditingController();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _confirmController = TextEditingController();

  bool _isLoading = false;

  final _authService = FakeAuthService();

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _confirmController.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    if (!_formKey.currentState!.validate()) return;

    if (_passwordController.text != _confirmController.text) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text("Passwords do not match")),
      );
      return;
    }

    setState(() => _isLoading = true);

    try {
      final user = UserModel(
        name: _nameController.text,
        email: _emailController.text,
        password: _passwordController.text,
      );

      await _authService.register(user);

      if (!mounted) return;

      Navigator.pushReplacement(
        context,
        MaterialPageRoute(
          builder: (_) => const SuccessScreen(),
        ),
      );
    } catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(e.toString())),
      );
    } finally {
      if (mounted) {
        setState(() => _isLoading = false);
```

```
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Register")),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Form(
          key: _formKey,
          child: Column(
            children: [
              TextFormField(
                controller: _nameController,
                decoration: const InputDecoration(labelText: "Name"),
                validator: Validators.required,
              ),
              TextFormField(
                controller: _emailController,
                decoration: const InputDecoration(labelText: "Email"),
                validator: Validators.email,
              ),
              TextFormField(
                controller: _passwordController,
                decoration: const InputDecoration(labelText: "Password"),
                obscureText: true,
                validator: Validators.password,
              ),
              TextFormField(
                controller: _confirmController,
                decoration:
                    const InputDecoration(labelText: "Confirm Password"),
                obscureText: true,
                validator: Validators.required,
              ),
              const SizedBox(height: 20),
              _isLoading
                  ? const CircularProgressIndicator()
                  : ElevatedButton(
                      onPressed: _isLoading ? null : _submit,
                      child: const Text("Register"),
                    ),
            ],
          ),
        ),
      ),
    );
  }
}
```

◈ BƯỚC 7 – SUCCESS SCREEN

```
import 'package:flutter/material.dart';

class SuccessScreen extends StatelessWidget {
  const SuccessScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Text(
          "Registration Successful!",
```


PhucPT10@fpt.edu.vn

```
            style: Theme.of(context).textTheme.headlineMedium,
          ),
        ),
      );
    }
}
```

VII. KIỂM TRA CÁC TRƯỜNG HỢP

Trường hợp	Kết quả mong đợi
Bỏ trống	Hiện lỗi
Email sai	Hiện lỗi
Password ngắn	Hiện lỗi
Confirm sai	SnackBar
Email test@error.com	Hiện lỗi từ service
Submit 2 lần	Không được

VIII. LỖI THƯỜNG GẶP

❗ Quên dispose controller
❗ Không kiểm tra mounted sau await
❗ Không disable button khi loading
❗ Không dùng try-catch

IX. MẸO & TIPS

💡 Luôn kiểm tra `if (!mounted) return;`
💡 Không navigate khi widget đã dispose
💡 Không dùng controller làm source of truth
💡 Tách validator ra file riêng
💡 Không gọi API trong validator

🎯 KẾT QUẢ SAU DỰ ÁN

Sinh viên sẽ:

Hiểu full vòng đời form
Biết viết submit flow chuẩn
Biết xử lý loading & error
Biết cách tổ chức project
Sẵn sàng nâng lên Clean Architecture

.

PhucPT10@fpt.edu.vn

I. FORM & CONTROLLERS

Thuật ngữ	Định nghĩa đơn giản	Giải thích chuyên sâu / Ghi chú
Form	Widget quản lý nhiều FormField	Hoạt động như một container quản lý trạng thái validation tập trung
FormState	Trạng thái nội bộ của Form	Gọi validate(), save(), reset()
GlobalKey<FormState>	Key dùng truy cập FormState	Cho phép thao tác Form từ bên ngoài widget
FormField	Widget con của Form	Ví dụ: TextFormField
TextFormField	Ô nhập liệu có validator	Kết hợp TextField + FormField
TextEditingController	Controller quản lý text	Giữ giá trị, selection, lắng nghe thay đổi
TextEditingValue	Giá trị nội bộ của controller	Gồm text + cursor position
ChangeNotifier	Cơ chế thông báo thay đổi	TextEditingController kế thừa từ đây
dispose()	Hủy tài nguyên	Phải gọi để tránh memory leak
mounted	Trạng thái widget còn tồn tại	Kiểm tra trước khi setState sau async

II. VALIDATION & ERRORS

Thuật ngữ	Định nghĩa đơn giản	Giải thích chuyên sâu
Validation	Kiểm tra dữ liệu hợp lệ	Có thể ở client hoặc server
Client-side validation	Kiểm tra trên app	Nhanh, UX tốt nhưng không bảo mật
Server-side validation	Kiểm tra backend	Bắt buộc để bảo mật
validator()	Hàm kiểm tra field	Phải synchronous
Async validation	Kiểm tra bằng API	Không nên đặt trong validator()
Regex	Biểu thức chính quy	Dùng kiểm tra email, phone
Required field	Trường bắt buộc	Không được để trống
Error message	Thông báo lỗi	Phải rõ ràng và thân thiện
Error mapping	Ánh xạ mã lỗi sang message	Không show raw backend message
Business error	Lỗi nghiệp vụ	Ví dụ: Email đã tồn tại
Network error	Lỗi kết nối	Timeout, mất mạng
Exception	Lỗi được throw	Phải được bắt bằng try-catch

III. SUBMIT FLOW & STATE

Thuật ngữ	Định nghĩa đơn giản	Giải thích chuyên sâu
Submit	Gửi dữ liệu	Thường gọi API
Submit Flow	Quy trình gửi dữ liệu	Bao gồm validate → loading → success/error
Loading state	Trạng thái đang xử lý	Phải hiển thị cho user
Double submit	Submit nhiều lần	Gây duplicate request
Disable button	Vô hiệu hóa nút	Ngăn double submit
State machine	Máy trạng thái	Submit flow thực chất là state machine
Idle state	Trạng thái ban đầu	Chưa submit
Success state	Thành công	Điều hướng hoặc hiển thị thông báo
Error state	Thất bại	Hiển thị lỗi
Retry	Thử lại	Sau khi fail

PhucPT10@fpt.edu.vn

Async/Await	Lập trình bất đồng bộ	Xử lý API
Future	Giá trị trả về sau	Đại diện cho kết quả async
try-catch	Bắt lỗi	Ngăn crash

IV. DATA & ARCHITECTURE

Thuật ngữ	Định nghĩa	Giải thích chuyên sâu
Model	Lớp dữ liệu	Đại diện cấu trúc dữ liệu
DTO	Data Transfer Object	Dùng khi gửi/nhận API
Repository	Tầng truy cập dữ liệu	Gọi API hoặc database
Separation of Concerns	Tách trách nhiệm	UI ≠ Business ≠ Data
Clean Architecture	Kiến trúc phân tầng	Tách Presentation / Domain / Data
ViewModel	Lớp quản lý trạng thái UI	Dùng trong kiến trúc nâng cao
Idempotency	Tính không lặp tác dụng	Submit nhiều lần không gây lỗi
Draft saving	Lưu tạm dữ liệu	Dùng trong form dài
Multi-step form	Form nhiều bước	Lưu state chung

V. UX & PRODUCTION CONCEPTS

Thuật ngữ	Định nghĩa	Giải thích
UX (User Experience)	Trải nghiệm người dùng	Validation ảnh hưởng lớn đến UX
Realtime validation	Kiểm tra ngay khi gõ	Có thể gây khó chịu nếu quá sớm
Hybrid validation	Kết hợp realtime + submit	Pattern phổ biến production
Debounce	Trì hoãn xử lý	Tránh spam API
Throttling	Giới hạn tần suất	Tương tự debounce
SnackBar	Thông báo tạm thời	Hiển thị lỗi chung
ScaffoldMessenger	Hiển thị SnackBar	Cách chuẩn hiện nay
Navigation	Điều hướng màn hình	Sau submit thành công

NHỮNG THUẬT NGỮ SINH VIÊN PHẢI THỰC SỰ HIỂU SÂU

FormState
TextEditingController
Validation vs Business Validation
Submit Flow
Loading State
State Machine
Exception Handling
Separation of Concerns

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

🔎 PROMPT 1 — Phân tích Form như một Data Pipeline

Phân tích Form trong Flutter dưới góc nhìn Data Pipeline.
Dữ liệu đi qua những giai đoạn nào từ lúc người dùng nhập đến khi lưu database?
Những điểm nào có thể gây lỗi hoặc mất dữ liệu?
Đề xuất kiến trúc giúp giảm rủi ro.

🎯 Mục tiêu tư duy:
Hiểu Form không chỉ là UI
Nhận diện validation layer – transformation – submission layer

🔎 PROMPT 2 — So sánh Controller vs State Management

So sánh việc quản lý dữ liệu form bằng TextEditingController với việc sử dụng State Management (Bloc, Riverpod, Provider).
Trong dự án lớn, phương án nào bền vững hơn? Vì sao?

🎯 Mục tiêu:
Phân tích ưu – nhược điểm
Hiểu giới hạn của controller

🔎 PROMPT 3 — Thiết kế Submit Flow như State Machine

Thiết kế Submit Flow của một form đăng ký phức tạp dưới dạng state machine.
Xác định rõ các trạng thái và transition.
Phân tích điều gì xảy ra nếu thiếu một trạng thái.

🎯 Mục tiêu:
Nhận diện state rõ ràng
Tránh logic mơ hồ

🔎 PROMPT 4 — Phân tích rủi ro Double Submit trong hệ thống tài chính

Phân tích rủi ro khi người dùng submit nhiều lần trong hệ thống ngân hàng hoặc thanh toán.
Frontend có thể xử lý những gì? Backend cần làm gì?
Đánh giá mức độ nghiêm trọng nếu không xử lý đúng.

🎯 Mục tiêu:
Hiểu idempotency
Tư duy hệ thống chứ không chỉ UI

🔎 PROMPT 5 — Đánh giá chiến lược Validation UX

So sánh ba chiến lược validation:
Validate realtime
Validate khi submit
Hybrid validation
Trong từng bối cảnh (e-commerce, ngân hàng, mạng xã hội), chiến lược nào phù hợp hơn? Vì sao?

🎯 Mục tiêu:
Không có 1 cách đúng duy nhất
Biết chọn chiến lược theo ngữ cảnh

🔎 PROMPT 6 — Phân tích vấn đề Async Validation

Nếu form cần kiểm tra email tồn tại bằng API, hãy phân tích các vấn đề có thể xảy ra:
Spam API

PhucPT10@fpt.edu.vn

Race condition
UI lag

Đề xuất giải pháp kiến trúc để xử lý.

🎯 Mục tiêu:
Nhận diện race condition
Hiểu debounce và cancellation

🔎 PROMPT 7 — Refactor Form sang Clean Architecture

Lấy một form đăng ký cơ bản và phân tích cách refactor nó sang Clean Architecture.
Xác định rõ Presentation – Domain – Data layer.
Điều gì thay đổi trong cách viết code?

🎯 Mục tiêu:
Hiểu separation of concerns
Thấy rõ giới hạn của form truyền thống

🔎 PROMPT 8 — Đánh giá Error Handling Strategy

Thiết kế hệ thống phân loại lỗi cho một form production-ready.
Phân loại:
Validation error
Business error
Network error
System error

Vì sao việc phân loại này quan trọng cho UX?

🎯 Mục tiêu:
Hiểu error không phải chỉ là String
Nhìn dưới góc nhìn hệ thống

🔎 PROMPT 9 — Phân tích Multi-step Form trong dự án lớn

Phân tích kiến trúc quản lý state của một multi-step form dài (ví dụ: đăng ký vay vốn).
So sánh:
Lưu state trong controller
Lưu state trong model chung
Lưu state trong global state

Phương án nào mở rộng tốt nhất?

🎯 Mục tiêu:
Tư duy mở rộng hệ thống
Đánh giá khả năng maintain

🔎 PROMPT 10 — Đánh giá khả năng Test của Form

Phân tích mức độ testable của một form truyền thống sử dụng StatefulWidget + Controller.
Những phần nào khó test?
Làm thế nào để cải thiện khả năng unit test và widget test?

🎯 Mục tiêu:
Hiểu tầm quan trọng của testability
Nhận diện tight coupling

PhucPT10@fpt.edu.vn

1 Thiết kế Form Module dùng lại trong toàn hệ thống

Nếu một hệ thống lớn có hàng chục form (đăng ký, thanh toán, profile, admin…), làm thế nào để thiết kế một Form Module reusable?
Cần chuẩn hóa những gì?
Làm sao tránh duplicate validation logic?

🎯 Trọng tâm:
- Reusable architecture
- Form abstraction
- Validation centralization

2 Chuẩn hóa Validation giữa Frontend và Backend

Trong dự án có nhiều team (mobile + backend), làm thế nào để đảm bảo rule validation luôn đồng bộ?
Nếu backend thay đổi rule, frontend xử lý thế nào?

🎯 Trọng tâm:
- API contract
- Versioning
- Shared schema

3 Thiết kế Async Validation không gây Spam API

Với field kiểm tra username/email tồn tại, làm thế nào để:
- Tránh spam API
- Tránh race condition
- Tránh lag UI

Thiết kế luồng xử lý tối ưu cho production.

🎯 Trọng tâm:
- Debounce
- Cancel previous request
- State isolation

4 Xử lý Double Submit trong hệ thống thanh toán

Trong hệ thống thanh toán thực tế, nếu user submit nhiều lần, hậu quả gì xảy ra?
Frontend và Backend cần phối hợp như thế nào để đảm bảo idempotency?

🎯 Trọng tâm:
- Idempotency key
- Transaction control
- Concurrency

5 Quản lý Multi-step Form dài trong dự án Enterprise

Với form đăng ký vay vốn 10 bước, làm thế nào để:
- Không mất dữ liệu khi crash
- Cho phép resume
- Tối ưu hiệu năng

Đề xuất kiến trúc phù hợp.

🎯 Trọng tâm:
- Draft saving
- Local storage
- State persistence

PhucPT10@fpt.edu.vn

6 Phân tích Performance của Form phức tạp

Form có 30+ field và validation phức tạp có thể gây giật lag.
Phân tích nguyên nhân và đề xuất cách tối ưu.

🎯 Trọng tâm:

- Rebuild control
- Controller management
- Lazy validation

7 Thiết kế Error Handling Strategy chuẩn doanh nghiệp

Xây dựng hệ thống phân loại lỗi chuẩn:

- UI error
- Business error
- Network error
- System error

Làm sao để mapping error rõ ràng và maintainable?

🎯 Trọng tâm:

- Error model
- Error mapper
- Localization

8 Logging & Monitoring Submit Flow trong Production

Trong môi trường production, form submission cần được log những gì?
Làm sao để phân tích tỷ lệ fail, timeout, abandon rate?

🎯 Trọng tâm:

- Analytics
- Crash reporting
- Observability

9 Test Strategy cho Form trong dự án lớn

Thiết kế chiến lược test cho:

- Validation logic
- Submit use case
- Widget test

Những phần nào nên unit test? Những phần nào nên integration test?

🎯 Trọng tâm:

- Testability
- Mock repository
- Separation of concerns

10 Bảo mật dữ liệu trong Form

Với form chứa thông tin nhạy cảm (password, OTP, tài khoản ngân hàng), cần xử lý:

- Logging
- Memory
- Autofill
- Clipboard

Những rủi ro bảo mật nào thường bị bỏ qua?

🎯 Trọng tâm:

- Secure input
- Không log sensitive data
- Clear controller

PhucPT10@fpt.edu.vn

1 1 Refactor Form sang Clean Architecture

Phân tích cách tách:

- Presentation
- Domain
- Data

So sánh maintainability trước và sau khi refactor.

🎯 Trọng tâm:

- Use case
- Repository pattern
- Dependency inversion

1 2 So sánh Form truyền thống vs Reactive Form

So sánh:

- Form + setState
- Bloc-based form
- Riverpod-based form

Khi nào nên chọn phương án nào?

🎯 Trọng tâm:

- Scalability
- Testability
- Complexity trade-off

1 3 UX Research cho Validation Strategy

Phân tích ảnh hưởng của:

- Realtime validation
- Validate on submit
- Hybrid

Dựa trên loại ứng dụng (Fintech vs Social App), chọn chiến lược phù hợp.

🎯 Trọng tâm:

- UX psychology
- Friction vs safety

1 4 Offline-first Strategy cho Form

Nếu ứng dụng hoạt động offline (logistics, field survey), submit flow cần thiết kế thế nào?
Làm sao queue request khi có mạng lại?

🎯 Trọng tâm:

- Local caching
- Sync mechanism
- Conflict resolution

1 5 Code Review Checklist cho Form Production

Xây dựng checklist code review dành riêng cho Form trong dự án lớn.
Bao gồm:

- Memory
- Error handling
- Validation
- UX
- Security

Điều gì thường bị junior bỏ sót?

PhucPT10@fpt.edu.vn

🎯 Trọng tâm:
Engineering quality
Risk prevention
Maintainability

PhucPT10@fpt.edu.vn